Talon: A WoW Threads Library

Programmer's Guide To Addon Development Using Asynchronous Multithreading

Abstract

This document is a programmer's guide for Talon, the code name for WoWThreads, an asynchronous, non-preemptive thread library for World of Warcraft addon developers. The guide describes some basic threading concepts and definitions and then documents each of the library's public API entry points.

Preface

```
If your only tool is a hammer, all problems look like nails
      -- Abraham Maslow

I've never met a young engineer who didn't think threads were great.
I've never met an older engineer who didn't shun them like the plague.
      -- Me
```

This work was motivated by a number of factors. The first, and most immediate, was to find a way to fix a problem when trying to add floating combat text to one of my [personal] addons. I needed a more flexible way to control the spacing between lines of text. It worked. My application didn't run faster (that wasn't the goal). In fact, I needed to slow down the rate at which frames of text scrolled up the screen. WoW threads offer several services that allow a programmer to adjust yield and delay times and this was exactly what I needed.

Second, most of my professional software career was spent designing multithread libraries and multithreaded applications. I thought "well, it was straight forward getting a minimal thread facility working using Blizzard's addon API", how much more difficult would it be to write a more functional one? I sank a couple of months into the project, found that it wasn't particularly difficult, and am satisfied with its functionality. But, and I can't say this strongly enough, I'm not convinced threading brings much to an addon developer's table. Threading is sexy and it's way cool to have it actually work. But does threading make for better addons? I think in most cases the improvement will be marginal, but in some cases it may be significant to be sure. But for general purposes, no.

Third, and maybe most importantly, there seems to be more than a little confusion about threads – both what they are and for what they should be used. My hope is that this document will help clear up some of that confusion and maybe help developers understand more clearly how to take advantage of threading.

Michael Peterson, Missoula, MT

Contents

Preface 2
About the Author 30
Introduction 6
Asynchronous and Synchronous Execution Models 6
Multithreading – Preemptive and Non-preemptive threads 8
Preemptive Threads 9
Non-preemptive threads 10
Concurrency 10
The WoW Multithread Library API 11
Design Considerations 11
Patterns 11
Code Structure 12
Startup 14
The Thread 14
The Thread Handle 14
Thread Creation 15
The Thread Control Block (aka Dispatch Table) 15
Signaling – Thread-to-Thread Communication 16
Signals 16
API 16
Error Handling 16
Debugging 17
Thread Services 17
Create A Thread 18
Delay a Thread 18
Yield to another thread. 18
Get A Thread's Unique Identifier 19
Get a Return Value From another thread's Function 20
Signal Services 21
Send a Signal To Another Thread 21
Retrieve a Signal 22
Management Services 22

Initialize the WoWThreads Library. 22
Get the yield interval for a specified thread 23
Set the yield interval for the specified thread 23
Get the clock dispatcher interval 23
Get the number of threads in the running, suspended, dead, and normal states. 24
Get thread performance metrics 24
Appendix 25
Installation 25
TOC File 25
File Header 25
Examples 26
Demo Application – A Simple Combat Log 28
Threads In Perspective 30

Introduction

Every so often a post will appear on one of the WoW Forums suggesting that Blizzard would be better off if their program development philosophy would just get with the times and redesign the WoW client to support multithreading.

This would not be wise for all kinds of reasons, but what strikes me as fascinating is is the misconceptions of many of the thread proponents have about threads and how best they can/should be used. Perhaps the most profound misconception is that the value of multithreading an application resides in the performance improvement gained by parallel execution of the application's threads. Nothing could be further from the truth (see the two articles at the end of this document under Threads In Perspective. Actually, multithreading for parallel execution is not that common except on high end systems not infrequently built with special hardware. It turns out that arguably the most common use of multithreading is for event- and task-oriented applications.

Asynchronous and Synchronous Execution Models

So, what exactly do I mean by asynchronous execution? Consider the following analogy (not mine, I heard this many years ago). Here we go:

Suppose a chef needs to fulfill an order for eggs and toast. S/he could fulfill this order using one of three execution models:

1. **Synchronous Execution**:

The chef makes the eggs. When the eggs are done, the chef makes the toast. When the toast is done the chef serves the customer cold eggs along with hot toast. This model is commonly used by soon to be unemployed chefs, but also reflects traditional programming in which procedure calls proceed synchronously, one after the other.

2. **Asynchronous Execution – Type I**

Immediately after the chef starts the eggs cooking, s/he turns to other kitchen tasks like washing dishes. When the eggs are almost done, s/he starts the toast so that it completes when the eggs are done. S/he then serves the customer hot toast and hot eggs. Most of us work this way naturally. It just makes sense to turn to some other task while waiting for an in-progress task to complete. The idea of working on more than one task at a time is the *sine qua non* of asynchronous execution models.

3. **Asynchronous – Type II**

In this model, the chef hires two additional cooks; an egg cook and a toast cook. The chef then coordinates the two cooks such that the eggs and toast finish at the same time. As in the Type I model, the customer gets hot eggs and toast.

However, the chef has to synchronize the two cooks (for some reason, they don't communicate with each other and often block each other when, for example, they both need to use the sink). In general, the chef is now required to ensure the two cooks will be able to share resources without unduly interfering with each other. In addition, since the employer now has to provide a salary for the two additional cooks, the cost of the meal is correspondingly higher and the customer is charged accordingly. This model makes sense only when favored by the economies of scale.

In this analogy, the asynchronous Type II model might be thought of as *employee-oriented*, while the Type I model is better thought of as *task-oriented*. The key here is that both models are asynchronous, both are useful, and both have their places. In terms of software applications, the Type II model is unbeatable for large-scale, high-end, compute intensive applications that, and this is key, can be structured to run multiple threads simultaneously and independently on separate processors – or, per the analogy, giving each of the cooks their own kitchen – a very expensive but sometimes necessary proposition.

By contrast, the Type I model is particularly effective for software applications that must wait frequently for something to occur. In this model, a single-process thread

runs all the time but, for event-driven reasons, may start a new task. For example, think of the WoW client as the single-process thread that runs continuously. When an event occurs (say a bag update event or a combat event) the WoW client initiates a new task by passing that event (via the *OnEvent()* service) to your addon. In the absence of asynchronous execution, the WoW client must wait for the addon to complete its event processing and return. By contrast, a Type I addon can pass the event's payload to an asynchronous thread that processes the event data. Because the thread executes asynchronously, the WoW client can immediately return to to other tasks without having to wait for the event processing to complete.

As it happens, gaming is event-driven and largely task-oriented. You ought not be surprised, therefore, to learn that almost all android games, and some modern game engines[1] employ asynchronous, multithreading (coroutines) as their design center.

Multithreading – Preemptive and Non-preemptive threads

The concept of threads as an executable object can be confusing so I'd like first to clarify not only what threads are, but what they are best used for. To begin, a thread is nothing more than a data structure (a table in Lua) containing as one of its elements a single procedure call. Nominally, a thread, like a process, is a set of data (a stack, registers, instruction pointer, a symbol table, etc.,) and a sequence of executable code (a procedure call) able able to be *independently* scheduled and run. But there is an important distinction here. A process, unlike a thread, is composed of a single address space. Traditional threads, by contrast, contain no address space but instead share the address space of a process. The practical consequences of this distinction are that creating a new thread is far less expensive than creating a new process. Moreover, since threads within the same process share its address space, data produced by one thread is immediately available to all other threads in the process. This makes communicating between threads simple and fast.

[1] Unity uses coroutines and is also a component of approximately ½ of all mobile games in the market today.

At the same time, they are similar to processes in that, just as processes can execute independent of each other, threads are similarly able to *execute independently* of other threads in that process. But, there are some complicating factors and they arise from the nature of how the scheduler switches between threads. These switches can be preemptive or non-preemptive (also called cooperatively self scheduling).

Preemptive Threads

Thread preemption requires operating system support[2] and is very resource intensive. So unless the cost of a thread context switch can be offset by an increase in parallelism, programmers should avoid preemption as they would a plague. Why? Because the difficulties involved in managing preemptive threads are legion and arise from the observation that a thread can be preempted at any point in its execution stream. This means that if the operating system decides to switch to another thread while the current thread is updating your bank account, you could be left broke. But there are well-understood, albeit risky, techniques for dealing with this problem. In my experience, the most difficult problem arising from the indeterminacy of preemptive thread execution is debugging. Fixing a timing bug that reveals itself only intermittently can lead a programmer to madness.

A second difficulty was hinted at a few sentences previously. Programmers must serialize access to critical sections (e.g., shared data such as your bank account) and this means locking. The requirement for locking not only substantially compromises performance but is rife with problems such as race conditions, deadlocks, and other horrors. On August 14, 2003, for example, a race condition arose in General Electric Energy's Unix-based energy management system that caused a power outage affecting 55 million people. Once triggered, the bug stalled FirstEnergy's control room alarm system for over an hour. The failure deprived the system operators of

[2] Thread preemption requires explicit hardware support in order to assign threads to specific processors or cores. Most modern operating system have kernel services that are able to trigger the hardware primitives that mediate processor assignment.

both audio and visual alerts. So, when a series of faults caused by tree branches touching power lines in Ohio went unnoticed, a cascade of failures was the result. This cascade led to the most widespread blackout in North American history

Nevertheless, and this is really important, threads of this class can be made to run simultaneously. Performance is paramount in high-end, compute intensive application software and preemptively scheduled threads are one of the best ways to satisfy high performance requirements. Ironically, this class of threads is what many of us think of when discussions about multithreading arise yet it is the model least applicable to general software.

Non-preemptive threads

In the non-preemptive model a thread executes until it yields the processor to another thread or terminates. In either case, the scheduler then submits the next thread eligible for execution to the run queue.

Concurrency

Programmers can achieve concurrency in two ways: multitasking and simultaneously (a.k.a., in parallel). Given two tasks, concurrency occurs when a cook works on both tasks by switching from one to the other (*single-threaded*, *asynchronous execution*). The second way to achieve concurrency and is to assign a cook to each task but instead of having them work simultaneously, have them trade off. So, when the first cook reaches a point where s/he has to wait for the oven temperature to get to 400 degrees, s/he turns over the kitchen to the second cook (*non-preemptive*, *multithreaded asynchronous execution).* This model is common in event-driven applications because the tasks in these environmentes are often *wait-blocked*. For example, suppose a thread's task is to retrieve certain records from a table of records and the table is empty. In this example, the thread's task cannot be completed at least one record appears. The task is therefore wait-blocked.

This second model has become the most common strategy for achieving asynchronous, concurrent execution for programs whose tasks are subject to indeterminate waiting.

The WoW Multithread Library API

The WoW Multithread Library (code name, Talon) is a collection of services that enable WoW addon developers to incorporate multiple, asynchronous, non-preemptive threads into their addon. Talon threads can increase and addon's concurrency, reduce coding complexity[3] making debugging and suppport easier.

Design Considerations

Patterns

In keeping with the analogy of how eggs and toast are made (see above), Talon programmers assign a thread to make the eggs and a thread to make the toast. In so doing, they arrange for each thread to pause its execution at appropriate times so the other thread can do its work. In other words, Talon employs multiple threads each of which has a specific task to perform. If that task has to wait (e.g., the eggs to cook) it can yield to let other threads execute their tasks (clean the kitchen, sharpen knives, and … start cooking the toast).

For example, consider how Talon might handle WoW events: one thread may do nothing more that receive events from the *OnEvent()* service and pass them off to other threads for the actual event handling. As it happens, this design pattern is one of the most common in gaming applications. The design pattern is pretty straightforward. When tasks arise, say in the form of events, a receiver thread assigns the task to a handler thread coded for that specific event.

[3] In the formal sense: code complexity is often measured in terms of the McCabe Complexity Metric (also called Cyclomatic Complexity) and is a measure of the linearly independent code paths through a program's source code.

Here's a typical *OnEvent()* snippet in which Talon's *thread:sendSignal()* service is used to notify a combat event logger that a combat event has just arrived and made available (i.e., inserted into a table) to the logger.

```
local function OnEvent( self, event, ... )
      local eventPayload = {CombatLogGetCurrentEventInfo()}
      table.insert( eventTable, eventPayload )

      thread:sendSignal( cleu_h, SIG_WAKEUP )
      return
end   --- OnEvent() returns
```

In this pattern, when the event fires the event payload is delivered to the addon through the *OnEvent()* service. In the absence of Talon, the WoW client will not return from its *OnEvent()* function until the event processing is completed by the addon. With Talon, however, your addon can hand off the event processing to one or more [asynchronous] threads and return from *OnEvent()*, even though the logger thread has processed the event. This pattern is illustrated in the snippet above. The addon signals a thread (*cleu_h*) that an event is ready for processing. Once the signal was sent, the addon returns from the client's *OnEvent()* service leaving the logging thread to process and log the event.

This is probably the most common design pattern for using multiple threads to handle asynchronous events. More generally, this pattern is one of a small collection of standard problems in multithreaded programming called the producer-consumer pattern. In this snippet, the WoW client 'produces' an event which is 'consumed' by an addon. Asynchrony happens because the producer of the event doesn't have to process the event. It just hands the event to another thread and goes back to producing stuff.

Code Structure

The art and practice of multithreading an addon involves, in part, understanding the distinction between thread context and WoW client context. Code executed by a thread is said to be executing in a thread context. Code executed by the WoW client is said to be running in the client context. Why is this important?

As it happens, while all Talon services can be executed by a Talon thread, only some can also be executed in both the client and thread contexts. For example, in the snippet above, the *thread:sendSignal()* executes in the [WoW] client's context. On the other hand, the corresponding *thread:getSignal()* service cannot be executed by the WoW client because the WoW client is not programmed to receive Talon signals. Programmers must be aware of which services can be executed by client code, Talon code, or both.

Because ALL Talon services can run in a thread context, my practice is to create most of my addon's threads in a *main()* function[4] like so:

```
local function main()
      local signal = SIG_NONE

      <… initialize addon stuff …>
      <… create threads to process addon tasks …>
      <… do addon stuff …>

      while signal ~= SIG_RETURN do
            thread:yield()
            signal = thread:getSignal()
      end
end
local yieldInterval = 3.0     -- seconds
local main_h, result = thread:create( yieldInterval, main )
```

This is by no means required. A couple of thoughts on the snippet: The while-loop contains two thread services, *thread:yield()* and *thread:getSignal()*, that only execute in a thread context – in this case within the main thread, *main_h*. The purpose of the loop is prevent the main thread from exiting. If fact, it will not exit until another thread sends it the SIG_RETURN signal.

[4] I come from a C/C++ background and the use of *main()* seemed appropriate. However, Talon doesn't care. You can name such functions anything you want.

Startup

Talon is initialized and started when the *mgmt:initWoWThreads()* service is called. In the WoWThreads installatino, *mgmt:initWoWThreads()* is called from within the ADDON_LOADED event handler.

```
local function OnEvent( self, event, ... )
  local addonName = ...
  if event == "ADDON_LOADED" and addonName == "WoWThreads" then
            < … lots of other non-thread initializations …>
      mgmt:initWoWThreads()
  end
```

The *mgmt:initWoWThreads()* service, initializes the Thread Control Block and establishes and begins running the clock timer.

Threads

The thread function is the procedure executed by the thread and passed into the create thread call. Here is an example of a thread function definition:

```
local function threadFunc( )
   <do stuff>
end
local handle, result = thread:create( interval, threadFunc, … )
```

In common threading parlance, the *threadFunc* method is also (and often) referred to as a *start* or *action routine*. Also note that in this guide, a thread is a composite object consisting of its handle, *thread_h,* and its thread function, *threadFunc().*

The Thread Handle

The thread handle is a table of attributes and state variables that are used to uniquely identify a thread and manage its semantics. Its structure is very simple:

Member Element	*Description*
TH_EXECUTABLE	A reference to the executable image
TH_IDENTIFIER	The unique Id of the thread handle
TH_THREAD_ADDRESS	The hex-address of the thread's function.
TH_STATUS	"running", "suspended", "normal", "dead"
TH_FUNC_ARGS	Parameters for the thread's function.
TH_JOIN_RESULTS	The data returned by *threadExit()*

TH_DELAY_TICKS_REMAINING	Delay time remaining in clock ticks
TH_SIGNAL	A pending signal otherwise SIG_NONE
TH_INITIAL_TICKS	Time at which thread was created.
TH_ACCUM_YIELD_TIME	Accumulated yield time.
TH_YIELD_COUNT	The number of times thread yielded.
TH_DURATION_TICKS	Thread's yield duration in clock ticks.
TH_REMAINING_TICKS	Yield time remaining in clock ticks.

Thread Creation

When a thread is created its state variables (used by the dispatcher to control its execution) are stored in the thread handle. Upon successful completion the handle is returned to the caller.

The Thread Control Block (aka Dispatch Table)

The *Thread Control Block* (TCB) is a table of thread handles. When a thread is created, its handle is inserted into the TCB. When the thread's function is dispatched (i.e., executed), the thread either runs to completion or until it suspends itself via a call to *thread:yield()*. Upon yielding the thread enters the *suspended* state.

The scheduler runs during every clock interval (the default interval is the reciprocal of your computer's frame rate[5].). During this interval the dispatcher examines the state of each thread in its TCB and does the following:

1. For each thread in the "suspended" state, the dispatcher decrements its remaining tick count (TH_REMAINING_TICKS). When the tick count reaches 0, the original tick count is restored (TH_DURATION_TICKS) and the thread is submitted to the run queue. The start routine of the newly resumed thread will continue execution at the point where it yielded.

2. For each thread in the "running" or "normal" state no action is taken.

[5] i.e., 1/*GetFramerate()*. For a framerate of 60 FPS, this gives a tick interval of 0.0167 seconds, or about 17 ms.

3. For each thread in the "dead" (completed) state the dispatcher moves the expired thread to the *graveyard* (a table of thread handles whose coroutines are in the "dead" state). At some point later in time, the scheduler moves in and sets to nil the handle's start routine and the handle.

Signaling – Thread-to-Thread Communication

In Talon, signals are integer values used to communicate between threads[i].

Signals

This initial release supports only two meaningful signals. The semantics of each signal are described below

`SIG_RETURN`	Upon receipt of this signal the thread is to return from the thread's function thereby terminating the thread.
`SIG_WAKEUP`	Another thread wants you to wake up right now and get back to work[6]. The SIG_WAKEUP signal is processed by the Talon scheduler. When received, the scheduler decrements the signaled thread's remaining tick count to zero and inserts the thread into the run queue.

Signals are sent and received by the *thread:sendSignal()* and the *thread:getSignal()* services respectively.

API

Error Handling

Most, but not all, public functions (those described below) return two values, the second of which is a result table containing an error code, an error message, and a stack trace. For example, looking again at the *thread:create()* service, note that it returns two values, the thread handle and the result block. The result block is a

[6] For example, a thread waiting on an event, could use the SIG_WAKEUP signal to notify a logging thread that an event record is ready for logging. SIG_WAKEUP is used because the dispatcher will immediately submit the signaled thread to the run queue despite its remaining tick count.

table containing three elements: an integer success/fail code, an error message, and a stack trace.

```
local thread_h, result = thread:create( t, func )
if thread_h == nil then
      mf:postResult( result )
      return
end
```

The *mf:postResult()* is a Talon service provided to display the error message and the stack trace in a message window.

Debugging

The *errors:dbgPrinte()* service is a modified print command that prints the file name and line number where the statement is written. I use it as a replacement for the print command. For example, if *errors:dbgPrint()* is placed at line 110 in file "MyFile.lua", its output will be

```
[MyFile.lua:110]
```

Here's a more complete example. Let *local E = errors*. Now, were I to place *E:dbgPrint()* in MyFile.lua at line 110 along with the message, "*In myFunc()*"

```
E:dbgPrint("In myFunc()")
```

then when executed by the addon *E:dbgPrint()* will produce,

```
[myFile.lua:110] In myFunc()
```

Thread Services

Threads are largely understood as overkill when used as simple wrappers around normal functions. Threads are better suited for functions that must execute only when some condition or other arises such as, for example, events, player I/O, UI Updates, etc. Having threads that sleep (yield) between events lessens the load on the system.

Create A Thread

If thread creation is successful, the thread handle and a result table are returned and the thread's function started. If the thread creation fails, no new thread is created, the returned handle is nil, and an error code, message, and stack trace are returned in the result table.

This service executes in both client and thread contexts.

Prototype

```
thread_h, status = thread:create( yieldInterval, threadFunc, … )
```

Parameters

`yieldInterval`	The time (in seconds) a thread will remain suspended after a call to *thread:yield()*
`threadFunc`	The function to be executed. The POSIX standard refers to this as the thread's *start_routine*.
…	A variable parameter list. This list must be a single value or, if multiple values are required, they must be placed in a table.

Returns

`thread_h`	An opaque, unique thread identifier.
`status`	A 3 element table containing a SUCCESS or FAILED code, an error message, and a stack trace.

Yield to another thread.

Causes the calling thread to yield its execution to the next thread scheduled for execution. The calling thread will be subsequently [re]executed when its interval time expires (set by the time interval parameter in the thread creation service).

This service can only execute in a thread context.

Prototype

```
thread:yield()
```

Parameters

`None`

Returns

`None`

Usage Example

In this example, the calling thread (the one executing the while loop) continues in the while loop until it receives a SIG_RETURN signal. Immediately after completing its "addon work", *thread:yield()* is called allowing other threads to proceed.

```
local function func ( thread_h )
      local result = nil
      local signal = SIG_NONE

      while signal ~= SIG_RETURN do
            <addon stuff>
        thread:yield()
            signal = thread:getSignal()
      end
end
```

Delay a Thread

Causes the calling thread to suspend its execution for the specified number of seconds. This service can only execute in a thread context. In other words, it will not delay the WoW client.

Prototype

```
thread:delay( seconds )
```

Parameters

`seconds`	The number of seconds the calling thread is to delay before resuming execution. Minimum is currently 0.02 seconds.

Returns

```
None
```

Usage Example

Get A Thread's Unique Identifier

Returns the calling thread's unique Id.

Prototype

```
thread_id, result = thread:getId()
```

Parameters

None

Returns

`thread_id`	integer value
`result`	A 3 element table, {SUCCESS/FAILURE, error message, Stack Trace}

Get a Return Value From another thread's Function

Allows the calling thread (e.g., a consumer thread) to block and wait for data from another thread (e.g., a producer thread).

Consider a producer thread whose function generates data to be used by other, consumer threads. Instead of returning the data in a global variable, the producer thread passes its result to *thread:exit()* instead of the normal *return*. For example, below is an example of threadExit() used in lieu of return.

```
local function generateData()
      data = getData()
      thread:exit( data ) - replaces "return data"
end
```

And, here's the creation of the generator thread:

```
local generator_h, result = thread:create( t, generateData )
```

Knowing the handle of the generator thread, other threads can "join" with the generator thread and retrieve the data generated by its thread function, *generateData()*.

```
local data, result = thread:join( generator_h )
```

The join function blocks the calling thread until the *generator_h* thread completes and returns its data to any (all) threads blocked and waiting for it to complete.

Both of these services can only execute in a thread context.

Prototypes

```
local returnValue, result = thread:join( producer_h )
```

```
thread:exit( returnValue  )
```

Parameters

`producer_h`	The handle of the thread from which the caller wishes to retrieve data (i.e., join).

Returns

`returnValue`	The value to be made available to all threads that have joined with this thread.

Signal Services

Talon currently supports three signals:

`SIG_NONE`	The default signal. Do nothing
`SIG_RETURN`	The signaled thread is to return immediately, i.e. exit the thread's function thus terminating the thread.
`SIG_WAKEUP`	This signal is provided for more efficient event handling. If your thread is an event handler (most of them?) you can create it with a looooooong yield interval. Then, when an event fires, signal the thread with SIG_WAKEUP. My combat logger thread has a yield time of two (2) hours. Between encounters the thread never runs. During an encounter, however, the thread is very, very busy because the event dispatcher is sending SIG_WAKEUP signals at machine gun rates.

Send a Signal To Another Thread

This service sends a signal to the specified thread. This service can execute in both client and thread contexts.

Prototype

```
local successful, result = thread:sendSignal( thread_h, signal )
```

Parameters

`thread_h`	Thread to receive the signal
`signal`	The signal to send to the targeted thread

Returns

`successful`	boolean. True if signal was delivered. The signal will remain pending until the target thread calls *thread:getSignal()*.False otherwise.

`result`	See Error Handling above

Retrieve a Signal

Retrieves a signal sent by another thread. If no signal is pending, SIG_NONE is returned. This service can only execute in a thread context.

Prototype

```
local signal, result = thread:getSignal()
```

Parameters

None

Returns

`Signal`	One of SIG_NONE, SIG_RETURN, or SIG_WAKEUP. If the function fails, signal will be nil
`Result`	See Error Handling

Usage Example

Management Services

Asynchronous programming, whether with preemptive or non-preemptive threads requires more than getting the syntax and the semantics correct. Tuning is a large part of getting an asynchronous program to run efficiently. These so-called management services are designed to give the programmer some control over the timing of program events. They allow an addon to query the runtime to obtain information about its threads and their execution environment.

Initialize the WoWThreads Library.

This service initializes the WoWThreads library. This service is commonly placed in the ADDON_LOADED event handler.

Prototype

```
mgmt:initWoWThreads()
```

Parameters

None

Returns

None

Get the yield interval for a specified thread

Prototype

```
yieldInterval, result = mgmt:getYieldInterval( thread_h )
```

Parameters

`thread_h`	thread handle

Returns

`yieldInterval`	The number of seconds, nil if error is encountered
`result`	Results block containing error message and stack trace if yieldInterval was nil

Set the yield interval for the specified thread

Prototype

```
isValid, result = mgmt:setYieldInterval( thread_h, yieldInterval )
```

Parameters

`thread_h`	Thread handle
`yieldInterval`	The number of seconds the thread is to wait before resuming execution.

Returns

`isValid`	Boolean true if no errors encountered. False otherwise
`result`	Results block containing error message and stack trace if yieldInterval was nil

Get the clock dispatcher interval

Prototype

```
mgmt:getClockInterval()
```

Parameters

None

Returns

`clockInterval`	The timer interval specified in Blizzard's C_After() service

Get the number of threads in the running, suspended, dead, and normal states.

Prototype

```
mgmt:getThreadCountByState()
```

Parameters

```
none
```

none

Returns

```
totalThreads          Total number of threads.
suspendedCount        Number of threads in the suspended state.
runningCount          Number of threads in the running state.
normalCount           Threads in "normal" state.
deadCount             Threads that have terminated.
```

Get thread performance metrics

Prototype

```
metricsTable, result = mgmt:getMetrics( thread_h )
```

Parameters

`thread_h`	Thread Handle

Returns

`threadState`	Thread state: "suspended", "running", "normal", "dead"
`avgYieldTime`	Average thread yield interval (seconds)
`ticksPerYield`	Average number of clock ticks per yield interval
`totalLifetime`	Total thread lifetime (seconds)
`yieldRatio`	Time spent in "suspended" state relative to total lifetime.
`runRatio`	Time spent in "running" state relative to total lifetime

Table of Metrics with the following values

Appendix

Installation

WoWThreads is a library, not an application. So, the installation is just a little different.

TOC File

1. First, unzip WoWThreads.zip to your Addons directory.
2. Next, move to your AddOn's directory and edit its .toc file by adding WoWThreads as a dependency. For example, I've bolded the dependencies line in the demo addon, combat logger's .toc

```
## Interface: 90005
## Title: CombatLogger
## Notes: Test of WoWThreads
## Author: Michael Peterson
## SavedVariablesPerCharacter:
## SavedVariables:
## Version: Demo - V1.0
## X-Expansion: Shadowlands
## 9 March, 2021
## Dependencies: WoWThreads
## ----------------------------------------
##    TEST APP: A SIMPLE COMBAT LOGGER
## ----------------------------------------
# Creates the publisher thread
Sources\Main.lua

# Reads and prints the formatted event strings as floating text
Sources\CleuPublishLogEntry.lua

# Formats CLEU events and inserts the formatted string into the
# event table
Sources\CleuGetEventStats.lua
```

That's all there is to it.

File Header

Here is the header declarations from the Main.lua file in the demo addon, CombatLogger. These declarations import all the symbols you'll need to run Talon out of the box.

```
-----------------------------------------------
```

```
-- Main.lua
-- AUTHOR: Michael Peterson
-- ORIGINAL DATE: 9 March, 2021
local _, CombatLogger = ...
CombatLogger.Main = {}
main = CombatLogger.Main

local fileName = "Main.lua"

local E = errors
local DEBUG = errors.DEBUG
local sprintf = _G.string.format

local SUCCESS = errors.STATUS_SUCCESS
local FAILURE = errors.STATUS_FAILURE

local SIG_NONE    = timer.SIG_NONE
local SIG_RETURN    = timer.SIG_RETURN
local SIG_WAKEUP    = timer.SIG_WAKEUP
local SIG_NUMBER_OF  = timer.SIG_WAKEUP

… Your Addon's specific symbols and code goes here
```

Examples

Hello World

What would a programmer's guide be without it's version of the inestimable "Hello World" program. To do this, this demo creates a greeter thread that, when signaled, prints "Hello World!" to the chat window. The code flow proceeds as follows:

1. A main thread is created.
2. The main thread creates the greeter thread.
3. The main thread signals the greeter thread to greet us.
4. The main thread enters a while loop waiting for SIG_RETURN
5. The greeter thread receives the signal and issues the greeting.
6. The greeter thread signals the main thread to exit (SIG_RETURN)
7. The main thread receives SIG_RETURN and terminates.

We begin by defining the greeter thread's function, *greet()*. The function's pattern, like almost all thread function patterns, is a while loop. In this loop the thread yields and waits to receive a SIG_WAKEUP signal When the signal arrives, greeter

prints "Hello World!", exits the loop, sends SIG_RETURN to the main thread (for reasons described below), and terminates.

So, let's get into the actual code: we begin by declaring the handles for our two threads, and then defining the *greet()* function.

```
local greeter_h = nil
local main_h = nil

local function greet()
      local signal = SIG_NONE

      while signal ~= SIG_WAKEUP do
            thread:yield()
            signal = thread:getSignal()
            if signal == SIG_WAKEUP then
                  print("Hello World!" )
            end
      end
      thread:signal( main_h, SIG_RETURN )
end
```

Now, we create the greeter thread (which will execute the *greet()* function). But, first recall that the call to *thread:yield()* must occur within a thread context which means, for all intents and purposes, that it must be called from another thread. My choice of names for these "generator" threads is … main(). So, before we can create the greeter thread, we must create its creator, the main thread.

Upon successful creation, the main thread immediately creates the greeter thread. The greeter thread then signals the greeter thread with SIG_WAKEUP which results in the greeter thread publishing "Hello World!" to the chat window and exiting the while loop thus terminating the thread. But after exiting the loop but before it returns (and terminates!), it sends the main thread a SIG_RETURN signal which causes it to exit the loop and terminate, thus terminating the application.

```
local function main()
      local signal = SIG_NONE
      local greeter_h, result = thread:create( yieldInterval, greet() )
      if greeter_h == nil then
            mf:postResult( result )
      end

      thread:signal( greeter_h, SIG_WAKEUP )
```

```
        while signal ~= SIG_RETURN do
              thread:yield()
              signal = thread:getSignal()
        end
end
local main_h, result = thread:create( yieldDuration, main )
if main_h == nil then
        mf:postResult( result )
end
```

Demo Application – A Simple Combat Log

This demo application is designed to illustrate the producer-consumer pattern using Talon threads in a working addon. In this example, the producer is that *OnEvent()* service. When the CLEU[7] fires, the *OnEvent()* services enters the event into a table, signals the consumer thread that one or more events are ready for processing, then exits.

Files

The code for this addon can be found in four files:

1. **Main.lua**: the code in this file initiates the *main()* function as the thread, *main_h* along with some logic that ensures that Talon is running. The body of main simply creates the publisher thread, *publisher_h*

2. **CleuGetEventStats.lua**: the code in this file, as you might guess, waits for the COMBAT_LOG_EVENT_UNFILTERED to fire. When the event is received, the *OnEvent()* handler formats a log-string (a text string) and inserts the string into a buffer called the *logEntryBuffer*. Note: the handler uses standard Lua table entry semantics meaning that the string is entered at the end of the buffer. This turns out to be important for reasons described in the next section.

3. **CleuPublishLogEntry**: the code in this file is defined by the publisher thread's function, *pub:publishLogEntry()*. This function is a classic thread-while loop. Here's its code:

[7] COMBAT_LOG_EVENT_UNFILTERED

```
function pub:publishLogEntry()
  local signal = SIG_NONE

  while signal ~= SIG_RETURN do
  local logEntry = event:logEntryBufferRemove()
  if logEntry ~= nil then
    writeFloatingText( logEntry  )
  end
  thread:yield()
  signal = thread:getSignal()
  end
  mf:postMsg( sprintf("publisher_h terminated.\n"))
end
```

The function *writeFloatingText()* is the service that actually writes the log entry, retrieved from the log buffer described above. Note the following: (1) The while loop terminates when the thread receives a SIG_RETURN signal. (2) The thread suspends itself (*thread:yield()*) once per iteration. This serves to smooth out the reading from the buffer and writing to the screen while allowing other portions of the code to execute. In the absence of the yield, the entire addon will hang. (3) Signals: two signals are of interest in this loop. First, if the thread receives a SIG_RETURN, the function terminates. Second if the thread receives a SIG_WAKEUP, the dispatcher decrements its remaining tick count to zero. In effect, this causes the thread to return immediately from *thread:yield()*. In this application, this is programmed to occur whenever the number of entries in the buffer exceeds a buffer limit (currently set to 3 – see CleuGetEventStats.lua and the LOG_BUFFER_LIMIT.

4. **CombatLogger.toc**: the load order is important. Main.lua is loaded first, the CleuPublishLogEntry.lua is loaded next, followed by CleuGetEventStats.lua

Startup Sequence

Because I use the addon as a test bed for WoWThreads, I start this by hand, i.e., I issue the following command: *Talon start* (I have this set as a macro). This command creates the main thread, *main_h*:

```
    local yieldInterval = 2.0
main_h, result = thread:create( yieldInterval, main )
if result[1] ~= SUCCESS then
```

```
            mf:postResult( result )
    return
  end
  talonActive = true
```

When the main_h thread is created, its function is immediately executed and its only purpose is to create the publisher thread, publisher_h.

```
    local function main()
    local result = {SUCCESS, nil, nil }

    -- This sets the yield interval for the publisher_h thread to 8 ticks
    -- of the Talon timer.
    local yieldInterval = mgmt:getClockInterval() * 8
    publisher_h, result = thread:create( yieldInterval,
          function()
                pub:publishLogEntry()
          end)
    if result[1] ~= SUCCESS then mf:postResult( result ) return end

    -- Wait for termination signal (SIG_RETURN)
    local signal = SIG_NONE
    while signal ~= SIG_RETURN do
  thread:yield()
  signal = thread:getSignal()
    end
```

Threads In Perspective

Threads are Evil: Parallelism meets programming reality

The Problem with Threads

About the Author

I began playing World of Warcraft shortly after its first release (Vanilla) and have leveled at least 1 character to max in every expansion since.

I hold a PhD in a field completely unrelated to programming (immunology) and as such confers absolutely no credibility insofar as programming is concerned. On the other hand, I am a retired software architect with the following experience:

- Architect and Lead developer of Digital Equipment Corporation's distributed UNIX-based off-line storage facility (Backup/Restore, Disaster Recovery, etc.,)

- Lead developer and architect for the disaster recovery facility[8] in the Windows NT operating system kernel.
- Wrote the first POSIX threads implementation for Linux[9].
- Author of DCE: A Guide To Developing Portable Applications[10]. This book evolved, in part, from my association with the development of Digital's Concert Multithread Architecture (CMA)[11]. CMA, and especially its lead developer, David Butenof. David was a formative influence on the development of POSIX threads[12].

As inferred earlier, one of the reasons I wrote a threads library for WoW addon development arises from what I believe is a good deal of confusion about threads – what they are and their good, bad, and ugly aspects. So, let's dive into these muddy waters and see if we can get a clearer understanding along with some perspective as to how multithreading a WoW addon might be just the thing your addon needs, but probably isn't.

---

[i] Signals in WoW ThreadLib are emphatically different from Unix signals. In Unix, signals are hard-coded integers[i] sent to other threads or processes that, when received, cause preordained actions to be taken. When a Unix process receives a SIG_INT signal, for example, the application terminates. In WoW ThreadLib no fixed, preprogrammed signal values (like SIG_INT, SIG_HUP, etc.,) exist.

---

[8] Formally known as ASR (Automated System Recovery)
[9] These were user-mode, preemptive threads.
[10] Peterson, Michael T., DCE: A Guide To Developing Portable Applications, McGraw-Hill, 1995
[11] A precurser model to the POSIX standard threads library.
[12] He was the author of one of the early Bibles of POSIX threads. You can click this link for further information: https://www.amazon.com/Programming-POSIX-Threads-David-Butenhof/dp/0201633922